

python import 机制与源码学习

目录

- 加载module的过程
 - finder的工作
 - loader的工作
- python系统代码追踪 (3.7)
 - 入口
 - 得到ModuleSpec
 - 得到Module
 - module_from_spec生成module
 - _init_module_attrs:
- Finders
 - PathFinder
- Loaders

官方文档: <https://docs.python.org/zh-cn/3/library/importlib.html>

参考blog: <https://www.biaodianfu.com/python-import.html>,

<http://www.seaung.cc/2020/03/21/python-loader/>

</>

Plain Text | 收起 ^

```
1 from types import ModuleType, MethodType
2 import importlib
3 from importlib.machinery import ModuleSpec
4 from importlib.abc import MetaPathFinder, Loader
5 from importlib.machinery import SourceFileLoader, ExtensionFileLoader,
  PathFinder
```

加载module的过程

sys.modules 是当前已加载的 module 的缓存 (dict)

在缓存未命中时, 需要先把module加载到 sys.modules , 这一工作由 finder、loader 等完成

- finder: 提供 find_spec 接口, 这个接口判断某个 module 能否被引入

- loader: 继承 `importlib.abc.Loader`, 提供 `exec_module` 方法, 这个方法将负责模块执行

finder的工作

系统在加载module时, 会遍历 `sys.meta_path` (list), 这个list里面有一些 finder

</>

Python | 收起 ^

```
1 for v in sys.meta_path:
2     print(v)
```

! 图片上传失败, 请重新上传

系统内置的Finder都是class, 以 class method的形式暴露接口

Python 会按照顺序使用 `sys.meta_path` 中的finder

- 期望finder能够返回一个ModuleSpec对象
- 若返回None, 则会遍历下一个 finder

(spec详见 `_init_module_attrs`)

可以自定义finder并使用:

</>

Python | 收起 ^

```
1 import sys
2 from importlib.abc import MetaPathFinder
3 from importlib.machinery import ModuleSpec
4
5 # 自定义finder 需要继承 MetaPathFinder
6 class NoSuchModuleFinder(MetaPathFinder):
7     def find_spec(self, fullname, path, target=None):
8         return ModuleSpec('nosuchmodule', None)
9
10 sys.meta_path = sys.meta_path + [NoSuchModuleFinder()]
11
12 # 可以用NoSuchModuleFinder找到任何module, 不管它是否存在, 因为它总是return一个
    ModuleSpec
13 import nosuchmodule
```

- 如果是 `import a.b`
- 会依次调用

```
1 find_spec("a", None, None)
2 find_spec("a.b", a.__path__, None)
```

loader的工作

finder返回的ModuleSpec应该有一个loader成员，它负责执行这一module

加载完毕后将module 加入 sys.modules

```
</> Python | 收起 ^

1 import sys
2 from contextlib import contextmanager
3 from types import ModuleType
4 import importlib
5 from importlib.machinery import ModuleSpec
6 from importlib.abc import MetaPathFinder, Loader
7
8 class MyModule(ModuleType):
9     def __init__(self, spec):
10         self.name = spec.name
11
12 class MyFinder(MetaPathFinder):
13     global padiff_loader
14     def find_spec(self, fullname, path, target=None):
15         if xxx:
16             return spec
17         return None
18
19 # exec_module 是一个 abstract method, 如果想要实现自定义的操作, 需要覆盖这个
    method
20 # 用于执行 module
21 class MyLoader(Loader):
22     # 如果exec_module被定义了, 那么这个函数必须要定义, 返回一个Module obj (可以
    return None 表示 default)
23     # Finder 返回的 spec 作为这里的输入
24     # 如果返回None, 表示使用 module_from_spec 的默认构造module的方法
25     def create_module(self, spec):
26         return MyModule(spec)
27     # create_module 的返回值作为这里的输入, 执行完之后会加入到sys.modules
28     def exec_module(self, module):
29         # 这里比较常规的做法是使用 compile
30         # obj = compile(poc_code, filename, 'exec', dont_inherit=True,
    optimize=-1)
```

```
31     # exec(obj, module.__dict__)
32     pass
33
34 sys.meta_path = [PaDiffFinder()] + sys.meta_path
```

python系统代码追踪 (3.7)

入口

import时, 会调用 `_find_and_load`

`/usr/local/lib/python3.7/importlib/_bootstrap.py`

</>

Python | 收起 ^

```
1 def _find_and_load(name, import_):
2     """Find and load the module."""
3     with _ModuleLockManager(name):
4         module = sys.modules.get(name, _NEEDS_LOADING)
5         if module is _NEEDS_LOADING:
6             # 入口
7             return _find_and_load_unlocked(name, import_)
8
9     if module is None:
10         message = ('import of {} halted; '
11                    'None in sys.modules'.format(name))
12         raise ModuleNotFoundError(message, name=name)
13
14     _lock_unlock_module(name)
15     return module
16
17
18 def _find_and_load_unlocked(name, import_):
19     path = None
20     parent = name.rpartition('.')[0]
21     if parent:
22         if parent not in sys.modules:
23             _call_with_frames_removed(import_, parent)
24         # Crazy side-effects!
25         if name in sys.modules:
26             return sys.modules[name]
27         parent_module = sys.modules[parent]
28         try:
29             path = parent_module.__path__
```

```
30     except AttributeError:
31         msg = (_ERR_MSG + '; {!r} is not a package').format(name,
parent)
32         raise ModuleNotFoundError(msg, name=name) from None
33     # 这里, 得到 ModuleSpec
34     spec = _find_spec(name, path)
35     if spec is None:
36         raise ModuleNotFoundError(_ERR_MSG.format(name), name=name)
37     else:
38         # 得到 Module
39         module = _load_unlocked(spec)
40     if parent:
41         # Set the module as an attribute on its parent.
42         parent_module = sys.modules[parent]
43         setattr(parent_module, name.rpartition('.')[2], module)
44     return module
```

得到ModuleSpec

/usr/local/lib/python3.7/importlib/_bootstrap.py

对于import外部module (比如 import paddle), 使用的是 <class
'_frozen_importlib_external.PathFinder'>

在 /usr/local/lib/python3.7/importlib/_bootstrap_external.py

</>

Python | 收起 ^

```
1 def _find_spec(name, path, target=None):
2     """Find a module's spec."""
3     meta_path = sys.meta_path
4     if meta_path is None:
5         # PyImport_Cleanup() is running or has been called.
6         raise ImportError("sys.meta_path is None, Python is likely "
7                             "shutting down")
8
9     if not meta_path:
10         _warnings.warn('sys.meta_path is empty', ImportWarning)
11
12     # We check sys.modules here for the reload case. While a passed-in
13     # target will usually indicate a reload there is no guarantee,
14     # whereas
15     # sys.modules provides one.
16     is_reload = name in sys.modules
```

```
17 # 遍历使用meta_path
18 for finder in meta_path:
19     with _ImportLockContext():
20         try:
21             find_spec = finder.find_spec
22         except AttributeError:
23             spec = _find_spec_legacy(finder, name, path)
24             if spec is None:
25                 continue
26         else:
27             # 调用 finder 的 find_spec
28             spec = find_spec(name, path, target)
29     if spec is not None:
30         # The parent import may have already imported this module.
31         if not is_reload and name in sys.modules:
32             module = sys.modules[name]
33             try:
34                 __spec__ = module.__spec__
35             except AttributeError:
36                 # We use the found spec since that is the one that
37                 # we would have used if the parent module hadn't
38                 # beaten us to the punch.
39                 return spec
40         else:
41             if __spec__ is None:
42                 return spec
43             else:
44                 return __spec__
45     else:
46         return spec
47 else:
48     return None
```

得到Module

</>

Python | 收起 ^

```
1 def _load_unlocked(spec):
2     # A helper for direct use by the import system.
3     if spec.loader is not None:
4         # not a namespace package
5         # 有loader 但没有 exec_module 时, 似乎不会进入
6         if not hasattr(spec.loader, 'exec_module'):
```

```
7         return _load_backward_compatible(spec)
8
9     # 使用这个接口生成module (空的)
10    module = module_from_spec(spec)
11    with _installed_safely(module):
12        if spec.loader is None:
13            if spec.submodule_search_locations is None:
14                raise ImportError('missing loader', name=spec.name)
15            # A namespace package so do nothing.
16        else:
17            # 执行module, 填充内容
18            spec.loader.exec_module(module)
19
20    # We don't ensure that the import-related module attributes get
21    # set in the sys.modules replacement case. Such modules are on
22    # their own.
23    return sys.modules[spec.name]
```

module_from_spec生成module

</>

Python | 收起 ^

```
1 def module_from_spec(spec):
2     """Create a module based on the provided spec."""
3     # Typically loaders will not implement create_module().
4     module = None
5
6     # 自定义exec_module后, 必须实现create_module
7     if hasattr(spec.loader, 'create_module'):
8         # If create_module() returns `None` then it means default
9         # module creation should be used.
10        module = spec.loader.create_module(spec)
11    elif hasattr(spec.loader, 'exec_module'):
12        raise ImportError('loaders that define exec_module() '
13                          'must also define create_module()')
14    if module is None:
15        # 默认行为, 构造空的Module obj (只有__name__)
16        module = _new_module(spec.name)
17    # 在构造完之后, 会把spec保存的属性复制过来, 所以初始化的时候不用担心
18    # 这里比较关键的是 __path__ 包必须有这个属性
19    _init_module_attrs(spec, module)
20    return module
21
```

```
22 def _new_module(name):
23     # 这里的sys就是 ModuleType
24     return type(sys)(name)
```

`_init_module_attrs:`

需要设置:

</>

Python | 收起 ^

```
1 module.__name__ = spec.name
  # 比如 paddle
2 module.__loader__ = spec.loader
  #
3 module.__package__ = spec.parent
  # 应该是 paddle (没有上级了)
4 module.__spec__ = spec
  #
5 module.__path__ = spec.submodule_search_locations  #
  'xxx/lib/python3.7/site-packages/paddle'
6 module.__file__ = spec.origin
  # 'xxx/lib/python3.7/site-packages/paddle/__init__.py'
7 module.__cached__ = spec.cached
  # pycache路径 'xxx/lib/python3.7/site-
  packages/paddle/__pycache__/__init__.cpython-37.pyc'
```

这里会检查 loader 是否为 None, 如果是 None (说明不是 module, 没 init), 但是有路径, 那么就会给一个 NamespaceLoader

</>

Python | 收起 ^

```
1 def _init_module_attrs(spec, module, *, override=False):
2     # The passed-in module may be not support attribute assignment,
3     # in which case we simply don't set the attributes.
4     # __name__
5     if (override or getattr(module, '__name__', None) is None):
6         try:
7             module.__name__ = spec.name
8         except AttributeError:
9             pass
10    # __loader__
11    if override or getattr(module, '__loader__', None) is None:
12        loader = spec.loader
13        # 到这里应该是有 loader 的
14        if loader is None:
```



```
15         # A backward compatibility hack.
16         if spec.submodule_search_locations is not None:
17             if _bootstrap_external is None:
18                 raise NotImplementedError
19             _NamespaceLoader = _bootstrap_external._NamespaceLoader
20
21             loader = _NamespaceLoader.__new__(_NamespaceLoader)
22             loader._path = spec.submodule_search_locations
23             spec.loader = loader
24
25             module.__file__ = None
26         try:
27             module.__loader__ = loader
28         except AttributeError:
29             pass
30     # __package__
31     if override or getattr(module, '__package__', None) is None:
32         try:
33             module.__package__ = spec.parent
34         except AttributeError:
35             pass
36     # __spec__
37     try:
38         module.__spec__ = spec
39     except AttributeError:
40         pass
41     # __path__
42     if override or getattr(module, '__path__', None) is None:
43         if spec.submodule_search_locations is not None:
44             try:
45                 module.__path__ = spec.submodule_search_locations
46             except AttributeError:
47                 pass
48     # __file__/__cached__
49     if spec.has_location:
50         if override or getattr(module, '__file__', None) is None:
51             try:
52                 module.__file__ = spec.origin
53             except AttributeError:
54                 pass
55
56         if override or getattr(module, '__cached__', None) is None:
57             if spec.cached is not None:
```

```
58         try:
59             module.__cached__ = spec.cached
60         except AttributeError:
61             pass
62     return module
```

Finders

PathFinder

这里会去sys.path_hooks里找一个finder，包括FileFinder等

</>

Plain Text | 收起 ^

```
1 [<class 'zipimport.zipimporter'>, <function FileFinder.path_hook.
   <locals>.path_hook_for_FileFinder at 0x7fad7078d1e0>]
```

</>

Python | 收起 ^

```
1 class PathFinder:
2
3     """Meta path finder for sys.path and package __path__ attributes."""
4
5     @classmethod
6     def invalidate_caches(cls):
7         """Call the invalidate_caches() method on all path entry finders
8         stored in sys.path_importer_caches (where implemented)."""
9         for name, finder in list(sys.path_importer_cache.items()):
10             if finder is None:
11                 del sys.path_importer_cache[name]
12             elif hasattr(finder, 'invalidate_caches'):
13                 finder.invalidate_caches()
14
15     @classmethod
16     def _path_hooks(cls, path):
17         """Search sys.path_hooks for a finder for 'path'."""
18         if sys.path_hooks is not None and not sys.path_hooks:
19             _warnings.warn('sys.path_hooks is empty', ImportWarning)
20         for hook in sys.path_hooks:
21             try:
22                 return hook(path)
23             except ImportError:
24                 continue
```

```
25         else:
26             return None
27
28     @classmethod
29     def _path_importer_cache(cls, path):
30         """Get the finder for the path entry from
31 sys.path_importer_cache.
32
33 If the path entry is not in the cache, find the appropriate
34 finder
35 and cache it. If no finder is available, store None.
36
37 """
38 if path == '':
39     try:
40         path = _os.getcwd()
41     except FileNotFoundError:
42         # Don't cache the failure as the cwd can easily change
43 to
44         # a valid directory later on.
45         return None
46
47     try:
48         finder = sys.path_importer_cache[path]
49     except KeyError:
50         finder = cls._path_hooks(path)
51         sys.path_importer_cache[path] = finder
52     return finder
53
54 @classmethod
55 def _legacy_get_spec(cls, fullname, finder):
56     # This would be a good place for a DeprecationWarning if
57     # we ended up going that route.
58     if hasattr(finder, 'find_loader'):
59         loader, portions = finder.find_loader(fullname)
60     else:
61         loader = finder.find_module(fullname)
62         portions = []
63     if loader is not None:
64         return _bootstrap.spec_from_loader(fullname, loader)
65     spec = _bootstrap.ModuleSpec(fullname, None)
66     spec.submodule_search_locations = portions
67     return spec
```

```

65     @classmethod
66     def _get_spec(cls, fullname, path, target=None):
67         """Find the loader or namespace_path for this module/package
        name."""
68         # If this ends up being a namespace package, namespace_path is
69         # the list of paths that will become its __path__
70         namespace_path = []
71         for entry in path:
72             if not isinstance(entry, (str, bytes)):
73                 continue
74             finder = cls._path_importer_cache(entry)
75             if finder is not None:
76                 if hasattr(finder, 'find_spec'):
77                     spec = finder.find_spec(fullname, target)
78                 else:
79                     spec = cls._legacy_get_spec(fullname, finder)
80                 if spec is None:
81                     continue
82                 if spec.loader is not None:
83                     return spec
84                 portions = spec.submodule_search_locations
85                 if portions is None:
86                     raise ImportError('spec missing loader')
87                 # This is possibly part of a namespace package.
88                 # Remember these path entries (if any) for when we
89                 # create a namespace package, and continue iterating
90                 # on path.
91                 namespace_path.extend(portions)
92         else:
93             spec = _bootstrap.ModuleSpec(fullname, None)
94             spec.submodule_search_locations = namespace_path
95             return spec
96
97     @classmethod
98     def find_spec(cls, fullname, path=None, target=None):
99         """Try to find a spec for 'fullname' on sys.path or 'path'.
100
101         The search is based on sys.path_hooks and
102         sys.path_importer_cache.
103         """
104         if path is None:
105             path = sys.path
106         spec = cls._get_spec(fullname, path, target)

```

```

106         if spec is None:
107             return None
108         elif spec.loader is None:
109             namespace_path = spec.submodule_search_locations
110             if namespace_path:
111                 # We found at least one namespace path. Return a spec
which
112                 # can create the namespace package.
113                 spec.origin = None
114                 spec.submodule_search_locations =
 NamespacePath(fullname, namespace_path, cls._get_spec)
115                 return spec
116             else:
117                 return None
118         else:
119             return spec
120
121     @classmethod
122     def find_module(cls, fullname, path=None):
123         """find the module on sys.path or 'path' based on sys.path_hooks
and
124         sys.path_importer_cache.
125
126         This method is deprecated. Use find_spec() instead.
127
128         """
129         spec = cls.find_spec(fullname, path)
130         if spec is None:
131             return None
132         return spec.loader

```

Loaders

最终一般都是SourceFileLoader

值得注意的是exex_module 和 get_code这两个函数

python3.7/importlib/_bootstrap_external.py

</>

Python | 收起 ^

```

1 class SourceFileLoader(FileLoader, SourceLoader):
2
3     """Concrete implementation of SourceLoader using the file system."""
4

```

```
5 def path_stats(self, path):
6     """Return the metadata for the path."""
7     st = _path_stat(path)
8     return {'mtime': st.st_mtime, 'size': st.st_size}
9
10 def _cache_bytecode(self, source_path, bytecode_path, data):
11     # Adapt between the two APIs
12     mode = _calc_mode(source_path)
13     return self.set_data(bytecode_path, data, _mode=mode)
14
15 def set_data(self, path, data, *, _mode=0o666):
16     """Write bytes data to a file."""
17     parent, filename = _path_split(path)
18     path_parts = []
19     # Figure out what directories are missing.
20     while parent and not _path_isdir(parent):
21         parent, part = _path_split(parent)
22         path_parts.append(part)
23     # Create needed directories.
24     for part in reversed(path_parts):
25         parent = _path_join(parent, part)
26         try:
27             _os.mkdir(parent)
28         except FileExistsError:
29             # Probably another Python process already created the
30             # dir.
31             continue
32         except OSError as exc:
33             # Could be a permission error, read-only filesystem:
34             # just forget
35             # about writing the data.
36             _bootstrap._verbose_message('could not create {!r}:
37             {!r}',
38                                         parent, exc)
39             return
40         try:
41             _write_atomic(path, data, _mode)
42             _bootstrap._verbose_message('created {!r}', path)
43         except OSError as exc:
44             # Same as above: just don't write the bytecode.
45             _bootstrap._verbose_message('could not create {!r}: {!r}',
46                                         path,
47                                         exc)
```

